

0.1 Tổng quan công nghệ và vai trò trong hệ thống

Trong đồ án, bài toán cần giải quyết là hiển thị giao thông trong không gian 3D dựa trên dữ liệu mô phỏng. Để đạt được mục tiêu này, hệ thống cần đồng thời đáp ứng ba nhóm yêu cầu chính: (i) mô phỏng giao thông có tính thực tế theo thời gian, (ii) trích xuất và (khi cần) điều khiển mô phỏng theo từng bước thời gian, và (iii) trực quan hoá kết quả dưới dạng 3D, có khả năng quan sát và tương tác.

Ngăn xếp công nghệ được lựa chọn bao gồm bốn thành phần chính:

- **SUMO** (Simulation of Urban Mobility): nền tảng mô phỏng giao thông đô thị mã nguồn mở, hỗ trợ mô phỏng vi mô theo từng phương tiện.
- **TraCI** (Traffic Control Interface): giao diện/protocol cho phép chương trình bên ngoài kết nối và điều khiển SUMO theo *timestep*.
- **Python**: lớp điều phối (orchestration) để khởi tạo mô phỏng, giao tiếp qua TraCI, và xuất dữ liệu theo một *chuẩn trao đổi* độc lập nền tảng hiển thị.
- **Unity**: môi trường dựng cảnh và hiển thị 3D theo thời gian thực, đảm nhận phần trực quan hoá phương tiện và hạ tầng.

Về mặt kiến trúc, luồng dữ liệu tổng quát có thể mô tả như sau: SUMO mô phỏng giao thông và cung cấp trạng thái đối tượng theo thời gian; Python sử dụng TraCI để lấy trạng thái này theo từng bước thời gian và xuất dữ liệu theo một định dạng chuẩn (hợp đồng dữ liệu) không phụ thuộc Unity; Unity (hoặc một tầng hiển thị khác trong tương lai) đọc dữ liệu chuẩn và thực hiện các bước chuyển đổi phục vụ việc render (ví dụ ánh xạ hệ toạ độ, nội suy).

TODO: Chèn hình kiến trúc tổng quan: **SUMO** ↔ **TraCI/Python** → **Unity**.
Gợi ý: vẽ dạng khối, ghi rõ dữ liệu trao đổi (ID xe, vị trí, tốc độ, hướng, trạng thái đèn).

Hình 0.1: Kiến trúc tổng quan hệ thống mô phỏng và hiển thị 3D

0.2 SUMO (Simulation of Urban Mobility)

0.2.1 Giới thiệu

SUMO là công cụ mô phỏng giao thông đô thị mã nguồn mở, được thiết kế để mô phỏng giao thông theo mô hình vi mô (microscopic simulation). Trong mô hình vi mô, mỗi phương tiện (xe máy, ô tô, xe buýt, ...) được xem như một thực thể có trạng thái riêng (vị trí, vận tốc, gia tốc, hướng di chuyển) và tuân theo các quy tắc

chuyển động/tuân thủ làn đường/điều khiển giao lộ. Cách tiếp cận này đặc biệt phù hợp khi mục tiêu là tái hiện chuyển động phương tiện một cách liên tục theo thời gian và phục vụ trực quan hoá.

Trong phạm vi đồ án, SUMO được sử dụng để tạo ra dữ liệu mô phỏng nhất quán theo thời gian. Thay vì chỉ xuất báo cáo thống kê tổng hợp, hệ thống cần lấy trạng thái chi tiết của từng phương tiện theo *timestep* để có thể hiển thị chuyển động trong Unity.

0.2.2 Lý do lựa chọn SUMO

SUMO được lựa chọn vì các lý do sau:

- **Phù hợp bài toán mô phỏng đô thị:** hỗ trợ mạng đường dạng đồ thị (nút–cạnh), mô phỏng làn đường, giao lộ, tín hiệu đèn.
- **Khả năng điều khiển thời gian thực:** kết hợp với TraCI cho phép chạy mô phỏng theo từng bước thời gian và truy vấn trạng thái tức thời.
- **Dễ tích hợp:** định dạng XML rõ ràng, hệ sinh thái công cụ hỗ trợ tạo mạng/route; phù hợp để Python xử lý và tự động hoá.
- **Mã nguồn mở:** thuận tiện cho mục tiêu học thuật và triển khai trong môi trường đồ án.

0.2.3 Các thành phần cấu hình và dữ liệu trong SUMO

Để chạy một kịch bản mô phỏng, SUMO thường cần ba nhóm tệp chính: tệp mô tả **mạng đường**, tệp mô tả **luồng/tuyến phương tiện**, và tệp **cấu hình mô phỏng**. Trong đồ án, các nhóm tệp này cũng là đầu vào/đầu ra quan trọng để đảm bảo tính lặp lại của thí nghiệm.

Mạng đường (network). Mạng đường thể hiện hạ tầng giao thông ở dạng đồ thị có hướng: nút (junction/node) và cạnh (edge). Tùy cách xây dựng mạng, có thể bắt đầu từ các tệp rời như `.nod.xml` và `.edg.xml`, sau đó biên dịch thành `.net.xml`. Tệp `.net.xml` là biểu diễn mạng ở mức chi tiết đủ để mô phỏng (bao gồm làn đường, toạ độ, kết nối ở nút giao, ...).

Luồng/tuyến (routes). Luồng phương tiện có thể được mô tả bằng tệp `.rou.xml`, trong đó định nghĩa tuyến đường (route), loại phương tiện (vehicle type) và lịch khởi hành (depart time). Khi mục tiêu là tái hiện giao thông có mật độ thay đổi theo thời gian, cách xây dựng route cần có khả năng điều chỉnh tham số như lưu lượng đầu vào, phân bố tuyến, hoặc kịch bản đặc thù (giờ cao điểm).

Cấu hình chạy (configuration). Tệp `.sumocfg` tập hợp các thành phần trên và chỉ định tham số mô phỏng: thời gian bắt đầu/kết thúc, độ dài bước thời gian

(*step-length*), chế độ xuất dữ liệu, Việc chuẩn hoá tham số trong tệp cấu hình giúp các thí nghiệm có thể chạy lại và so sánh.

Bảng 1: Đầu vào/đầu ra của SUMO trong hệ thống (tóm tắt)

Nhóm	Mô tả
Đầu vào	Mạng đường (.net.xml), luồng/tuyến (.rou.xml), cấu hình (.sumocfg)
Đầu ra	Trạng thái theo thời gian: danh sách phương tiện, vị trí (x, y), vận tốc, hướng; trạng thái đèn; thông tin edge/lane (nếu cần)

0.2.4 Các lựa chọn thay thế và đánh giá

Với bài toán mô phỏng giao thông, có thể xem xét nhiều công cụ khác nhau. Một số công cụ mô phỏng thương mại như Aimsun hoặc VISSIM có khả năng mô phỏng mạnh và nhiều module, tuy nhiên đi kèm hạn chế về giấy phép, chi phí và mức độ phù hợp trong bối cảnh triển khai đồ án. Ở chiều ngược lại, các công cụ mô phỏng vĩ mô (ví dụ MATSim) tập trung vào lưu lượng tổng thể và hành vi theo mô hình nhu cầu, không tối ưu cho việc lấy trạng thái chi tiết của từng phương tiện để hiển thị 3D liên tục. Ngoài ra, các môi trường 3D phục vụ xe tự hành (ví dụ CARLA) cung cấp mô phỏng hình ảnh và cảm biến, nhưng chi phí tích hợp và yêu cầu tài nguyên thường lớn hơn đáng kể so với mục tiêu hiển thị giao thông dựa trên dữ liệu mô phỏng.

0.3 TraCI (Traffic Control Interface)

0.3.1 Giới thiệu

TraCI là giao diện/protocol cho phép một chương trình bên ngoài (client) kết nối tới SUMO (server) để điều khiển mô phỏng theo từng bước thời gian. Thay vì để SUMO tự chạy độc lập và chỉ xuất log sau khi kết thúc, TraCI cho phép hệ thống:

- điều khiển tiến trình mô phỏng theo *timestep* (bước thời gian rời rạc),
- truy vấn trạng thái đối tượng ngay tại thời điểm đang chạy,
- thay đổi trạng thái/ra lệnh điều khiển (nếu kịch bản yêu cầu),
- đồng bộ dữ liệu mô phỏng với hệ thống hiển thị 3D.

Trong đồ án, TraCI là “cầu nối” quan trọng để Python có thể lấy dữ liệu theo thời gian thực và cung cấp cho Unity. Nhờ đó, Unity không cần đọc các tệp log hậu kỳ mà có thể cập nhật chuyển động theo nhịp mô phỏng.

0.3.2 Mô hình hoạt động client–server

Về nguyên lý, SUMO chạy như một server TraCI. Ứng dụng Python đóng vai trò client: khởi tạo kết nối, gửi lệnh `simulationStep` để tiến mô phỏng, sau đó truy vấn các đối tượng quan tâm (ví dụ danh sách xe đang tồn tại, vị trí của từng xe, trạng thái đèn tín hiệu). Mỗi vòng lặp tương ứng với một bước thời gian, từ đó hình thành luồng dữ liệu theo dạng chuỗi thời gian.

TODO: Chèn hình minh hoạ vòng lặp TraCI: `connect` → `simulationStep` → `get state` → `export/send to Unity`.

Hình 0.2: Vòng lặp điều khiển và thu thập dữ liệu bằng TraCI

0.3.3 Nhóm chức năng TraCI sử dụng trong đồ án

Tùy theo kịch bản và dữ liệu cần hiển thị, TraCI cung cấp nhiều nhóm hàm truy vấn/điều khiển. Trong đồ án hiển thị 3D, các nhóm chức năng thường dùng gồm:

- **simulation:** điều khiển tiến trình mô phỏng, lấy thời gian mô phỏng, số xe đang tồn tại.
- **vehicle:** lấy và cập nhật trạng thái xe (toạ độ, tốc độ, hướng), truy vấn tuyến hiện tại, làn hiện tại.
- **trafficlight** (nếu có): lấy/đặt pha đèn, thời gian còn lại của pha.
- **edge/lane:** lấy thông tin hạ tầng (độ dài, tốc độ giới hạn) và các thống kê cục bộ (nếu phục vụ hiển thị/đánh giá).

0.3.4 Lựa chọn thay thế

Một cách tiếp cận thay thế là chạy SUMO và xuất log sau mô phỏng (ví dụ dạng CSV/FCD), rồi để Unity đọc lại và phát lại như một đoạn phim. Cách này có ưu điểm đơn giản, dễ debug, nhưng hạn chế khả năng đồng bộ theo thời gian thực và khó mở rộng sang các tình huống cần can thiệp mô phỏng trong lúc chạy (ví dụ thay đổi chính sách đèn, điều tiết xe theo sự kiện). Do đó, TraCI phù hợp hơn với mục tiêu “kết nối mô phỏng–hiển thị” trong đồ án.

0.4 Python cho điều phối và xử lý dữ liệu

0.4.1 Vai trò trong hệ thống

Python đóng vai trò lớp điều phối giữa mô phỏng (SUMO) và hiển thị (Unity). Ở mức hệ thống, Python chịu trách nhiệm:

- khởi tạo phiên làm việc với SUMO (chạy SUMO/GUI, cấu hình cổng TraCI),
- điều khiển mô phỏng theo từng bước thời gian thông qua TraCI,
- thu thập dữ liệu cần thiết (phương tiện, trạng thái đèn, ...),
- chuẩn hoá dữ liệu theo **chuẩn trao đổi độc lập nền tảng** (lọc trường không cần thiết, thống nhất đơn vị/định dạng và cấu trúc bản ghi),
- xuất/gửi dữ liệu theo cơ chế trao đổi đã chọn.

So với việc viết toàn bộ lớp điều phối trong Unity/C#, Python mang lại lợi thế về tốc độ phát triển, khả năng thử nghiệm nhanh, và thuận tiện cho việc viết script tự động tạo dữ liệu (tạo map, tạo route, chạy hàng loạt kịch bản).

0.4.2 Chuẩn hoá theo hợp đồng dữ liệu độc lập nền tảng

Một mục tiêu quan trọng của hệ thống là tách biệt tầng mô phỏng/thu thập dữ liệu (Python) khỏi tầng hiển thị (Unity). Do đó, Python **không thực hiện các chuyển đổi đặc thù Unity** như ánh xạ toạ độ 2D của SUMO sang trục 3D của Unity hay chuyển đổi rotation theo quy ước engine. Thay vào đó, Python xuất dữ liệu theo một **hợp đồng dữ liệu chuẩn** (data contract) có thể tái sử dụng bởi bất kỳ nền tảng đồ hoạ nào.

Hợp đồng dữ liệu chuẩn cần thống nhất các nội dung tối thiểu sau:

- **Đơn vị đo:** ví dụ vị trí theo mét, thời gian theo giây, vận tốc theo m/s.
- **Hệ toạ độ nguồn:** giữ nguyên hệ toạ độ của SUMO (2D) và cung cấp mô tả/metadata cần thiết để tầng hiển thị tự ánh xạ.
- **Trường dữ liệu bắt buộc:** `vehicleId`, `t`, `x`, `y`, `speed`, `heading` (và các trường tùy chọn nếu cần).
- **Định dạng bản ghi:** cấu trúc dòng/bản ghi (CSV/JSON) và quy ước về dấu phân cách, encoding, thứ tự trường.

Nhờ cách thiết kế này, khi thay đổi nền tảng hiển thị (ví dụ chuyển từ Unity sang một engine khác), phần Python có thể giữ nguyên; chỉ cần xây dựng một tầng “adapter hiển thị” mới để đọc dữ liệu chuẩn và thực hiện các chuyển đổi phù hợp với engine.

0.4.3 Tổ chức xử lý theo bước thời gian

Một chiến lược phổ biến là Python chạy vòng lặp theo timestep: mỗi bước tiến mô phỏng một khoảng Δt , sau đó lấy trạng thái và gửi sang Unity. Trong các bước này, có thể áp dụng tối ưu như chỉ gửi các phương tiện đang tồn tại, hoặc chỉ gửi những thay đổi so với bước trước. Ngoài ra, để chuyển động mượt trong Unity, có

thể lựa chọn gửi dữ liệu ở tần suất thấp hơn và nội suy trong Unity.

0.4.4 Lựa chọn thay thế

Về mặt kỹ thuật, TraCI cũng hỗ trợ các client trong Java/C++ và có thể xây dựng lớp điều phối bằng các ngôn ngữ này. Tuy nhiên, với đồ án ứng dụng, Python là lựa chọn cân bằng giữa tính linh hoạt, khả năng xử lý dữ liệu và thời gian phát triển. Một lựa chọn khác là viết trực tiếp lớp giao tiếp trong Unity/C#, giúp giảm một tầng trung gian, nhưng làm tăng độ phức tạp triển khai và khó tái sử dụng các script xử lý dữ liệu/phân tích.

0.5 Unity cho hiển thị 3D và tương tác

0.5.1 Giới thiệu và lý do lựa chọn

Unity là nền tảng phát triển ứng dụng 3D thời gian thực, hỗ trợ dựng cảnh, điều khiển camera, chiếu sáng và xây dựng giao diện tương tác. Trong đồ án, Unity đảm nhiệm vai trò trực quan hoá giao thông 3D dựa trên dữ liệu mô phỏng từ SUMO. Việc hiển thị 3D cho phép người dùng quan sát trực quan mật độ xe, dòng xe qua nút giao, và hành vi di chuyển theo thời gian, từ đó hỗ trợ phân tích và trình bày kết quả.

Unity được lựa chọn vì các lý do chính: (i) hệ sinh thái phong phú cho dựng cảnh và xử lý chuyển động, (ii) khả năng cập nhật đối tượng theo thời gian thực, và (iii) phù hợp với yêu cầu triển khai nhanh trong đồ án.

0.5.2 Pipeline hiển thị đối tượng giao thông

Để hiển thị một mô phỏng giao thông trong Unity, hệ thống cần quản lý hai nhóm đối tượng: **hạ tầng** (đường, làn, nút giao) và **đối tượng động** (phương tiện, có thể mở rộng sang người đi bộ nếu kịch bản có). Một pipeline hiển thị thường gồm các bước:

- **Khởi tạo hạ tầng:** dựng mặt đường, làn đường và các điểm mốc dựa trên dữ liệu mạng (từ SUMO hoặc dữ liệu chuyển đổi trung gian).
- **Khởi tạo phương tiện:** tạo đối tượng xe theo danh sách ID, gán mô hình (mesh/prefab) và thuộc tính hiển thị.
- **Cập nhật theo thời gian:** tại mỗi bước, cập nhật vị trí và hướng xe dựa trên dữ liệu nhận từ Python.

Trong trường hợp số lượng xe lớn, một vấn đề quan trọng là hiệu năng. Một số kỹ thuật thường dùng gồm tái sử dụng đối tượng (object pooling), giảm số lần tạo/hủy đối tượng, hoặc chỉ hiển thị trong phạm vi quan sát.

0.5.3 Lựa chọn thay thế

Các engine 3D khác như Unreal Engine có khả năng đồ hoạ mạnh, tuy nhiên thường yêu cầu tài nguyên lớn và chu trình phát triển có thể nặng hơn. Các giải pháp WebGL (Three.js) thuận lợi cho triển khai web nhưng cần tự xây nhiều thành phần engine/asset và tối ưu hiệu năng. Vì vậy Unity là lựa chọn phù hợp cho mục tiêu trực quan hoá 3D trong đồ án.

0.6 Tích hợp SUMO–TraCI/Python–Unity

0.6.1 Phương thức trao đổi dữ liệu

Trong dự án, dữ liệu được truyền theo thời gian thực bằng cơ chế **TCP/IP**. Cụ thể, Python đóng vai trò tiến trình phía gửi (server) và Unity đóng vai trò phía nhận, liên tục đọc gói dữ liệu và cập nhật trạng thái hiển thị.

Cách tiếp cận truyền trực tiếp qua TCP/IP có các ưu điểm chính: độ trễ thấp, đồng bộ tốt với tiến trình mô phỏng theo từng bước thời gian, và phù hợp với mục tiêu hiển thị “đồng hồ thời gian” theo mô phỏng. Ngoài ra, vì dữ liệu được đóng gói theo hợp đồng dữ liệu chuẩn, tầng truyền TCP/IP cũng có thể tái sử dụng nếu thay Unity bằng một nền tảng hiển thị khác trong tương lai.

Dữ liệu được gửi theo chu kỳ **mỗi timestep**; kích thước trung bình mỗi gói khoảng **128KB**; cơ chế đóng gói và giải mã sử dụng **JSON**.

0.6.2 Schema dữ liệu tối thiểu

Để hiển thị chuyển động phương tiện, Unity tối thiểu cần biết: định danh xe, vị trí, hướng và vận tốc tại mỗi mốc thời gian. Ngoài ra, để phục vụ phân tích và mở rộng, có thể kèm theo thông tin làn/edge hoặc trạng thái đèn.

Lưu ý rằng schema này được thiết kế theo hướng **độc lập engine**: các trường phản ánh trạng thái mô phỏng và hình học nguồn. Các bước chuyển đổi sang hệ trục 3D, tỉ lệ hiển thị, cũng như nội suy theo frame sẽ do tầng hiển thị (Unity) đảm nhiệm.

Bảng 2: Schema dữ liệu tối thiểu trao đổi giữa Python và Unity

Trường dữ liệu	Ý nghĩa
vehicleId	Định danh phương tiện (duy nhất trong mô phỏng)
x, y	Toạ độ 2D trong hệ SUMO
speed	Vận tốc (m/s hoặc km/h theo quy ước)
(tùy chọn) laneId/edgeId	Làn/đoạn đường hiện tại để tô màu, thống kê, ...

0.6.3 Đồng bộ thời gian giữa timestep và frame

SUMO thường vận hành theo bước thời gian rời rạc (timestep) với độ dài Δt cố định. Trong khi đó Unity cập nhật theo số khung hình mỗi giây (FPS) và có thể thay đổi theo cấu hình máy. Điều này tạo ra bài toán đồng bộ: nếu Unity chỉ cập nhật vị trí xe đúng tại mỗi timestep, chuyển động có thể bị “giật” khi FPS cao. Vì vậy, có hai cách tiếp cận:

- **Cập nhật theo timestep:** đơn giản, đúng dữ liệu, phù hợp khi ưu tiên tính chính xác và dễ triển khai.
- **Nội suy theo frame:** Unity nhận hai trạng thái liên tiếp và nội suy vị trí/rotation cho các frame ở giữa, giúp chuyển động mượt hơn.

Trong dự án này, hệ thống lựa chọn **nội suy theo từng frame** ở tầng hiển thị (Unity) để đảm bảo chuyển động mượt và ổn định khi FPS cao.

Việc lựa chọn cách đồng bộ phụ thuộc vào mục tiêu của đồ án (chính xác tuyệt đối theo mô phỏng hay ưu tiên cảm nhận trực quan mượt mà) và quy mô số lượng xe hiển thị.

0.6.4 Tổng kết chương

Chương này đã trình bày các công nghệ chính được sử dụng trong đồ án gồm SUMO, TraCI, Python và Unity, cùng vai trò của từng thành phần trong kiến trúc tổng thể. Đồng thời, chương cũng phân tích cách tích hợp giữa mô phỏng và hiển thị 3D, đặc biệt là vấn đề trao đổi dữ liệu và đồng bộ thời gian, làm nền tảng cho phần thiết kế và triển khai hệ thống ở các chương sau.